

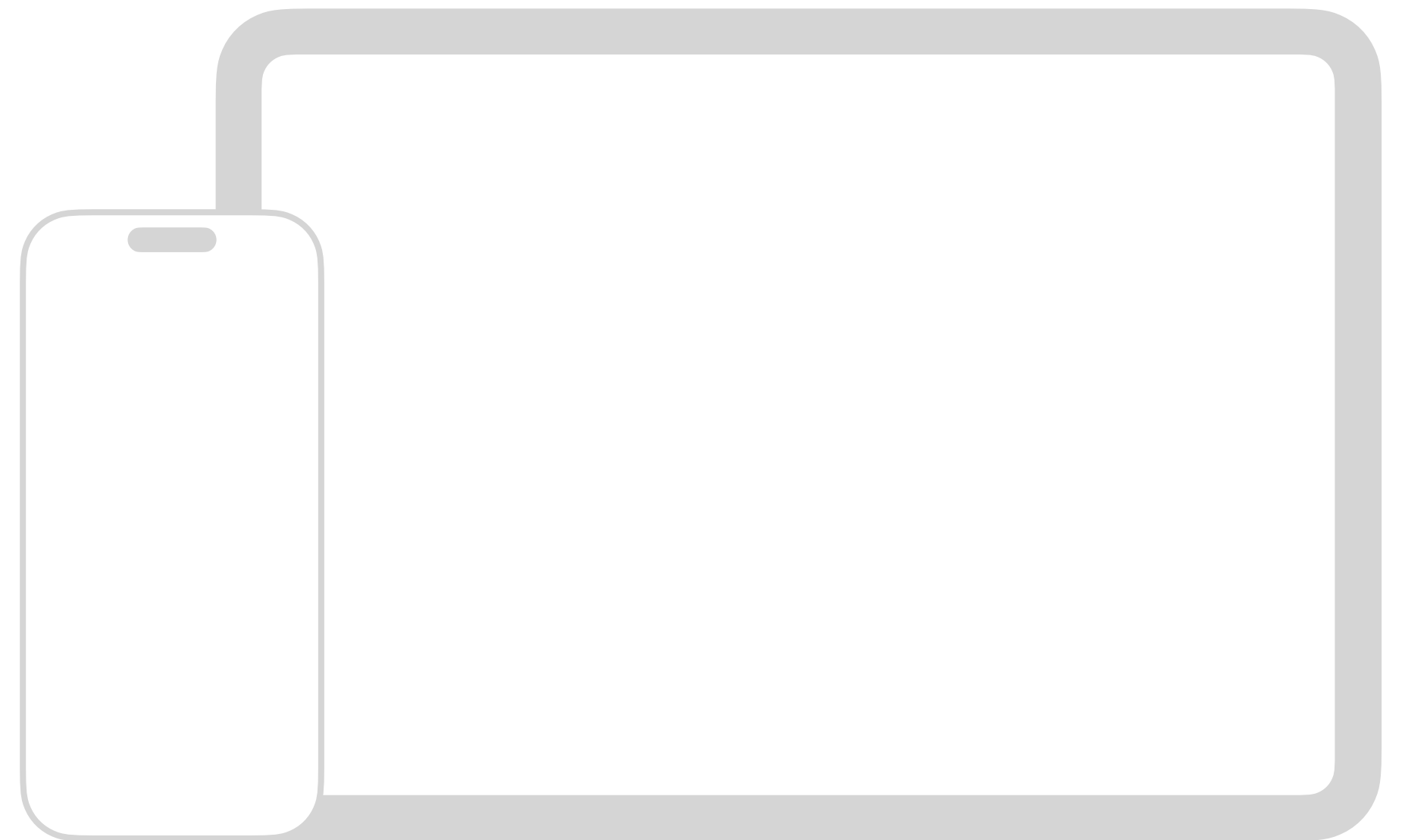
บทเรียนที่ 7-2

- การเรียกและดึงข้อมูลจาก API



iOS Application Development

วิศวกรรมคอมพิวเตอร์และเทคโนโลยีดิจิทัล จุฬาลงกรณ์มหาวิทยาลัย



โครงสร้างข้อมูล JSON

โครงสร้างข้อมูล JSON

```
{  
  "name": "Somchai",  
  "age": 19,  
  "skills": ["Swift", "iOS", "Design"]  
}
```

JSON (JavaScript Object Notation)

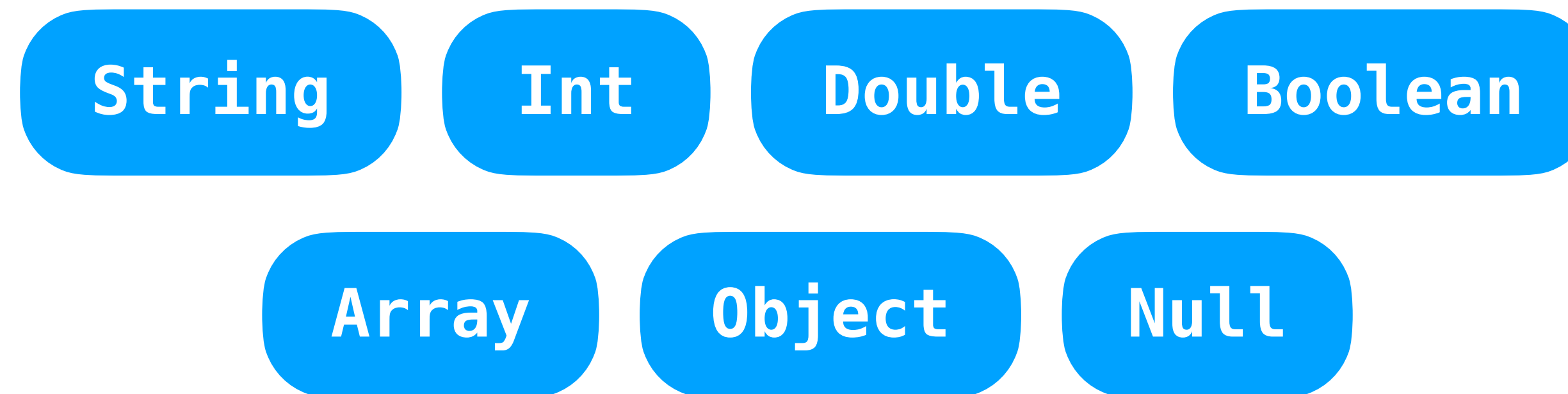
เป็นรูปแบบการแลกเปลี่ยนหรือรับส่งข้อมูลในระบบคอมพิวเตอร์หรือแอปพลิเคชัน เป็นที่นิยมโดยเฉพาะในงานด้านการทำ API ต่าง ๆ

JSON Object

```
{  
  "key": "value"  
}
```

มีรูปแบบการเก็บข้อมูลแบบ Key Value

ประเภทของข้อมูลที่ JSON รองรับ



การสร้าง JSON Object

{

```
"name": "Somchai",  
"age": 19,  
"skills": ["Swift", "iOS", "Design"]
```

}

การสร้าง JSON Object

```
{  
  "name": "Somchai",  
  "age": 19,  
  "skills": ["Swift", "iOS", "Design"]  
}
```

การสร้าง JSON Object

```
{  
  "name": "Somchai",  
  "age": 19,  
  "skills": ["Swift", "iOS", "Design"]  
}
```


การสร้าง JSON Object

```
{  
  "name": "Somchai",  
  "age": 19,  
  "skills": ["Swift", "iOS", "Design"]  
}
```

การสร้าง JSON Object

```
{  
    "name": "Somchai",  
    "age": 19,  
    "skills": ["Swift", "iOS", "Design"],  
    "address": {  
        "number": "999/9",  
        "road": "Ratchadamri Rd.",  
        "postalCode": 10330  
    }  
}
```

แปลงข้อมูล JSON เป็น Struct

Codable

โปรโตคอลที่ใช้สำหรับแปลงข้อมูล
ภายนอก (เช่น JSON) โดยอัตโนมัติ

แปลงข้อมูล JSON เป็น Struct

```
{  
    "id": 1,  
    "name": "John Doe",  
    "email": "john.doe@example.com"  
}
```

```
struct User: Codable {  
    let id: Int  
    let name: String  
    let email: String  
}
```

1 เพิ่ม protocol Codable ใน struct ที่สร้าง


```
let decoder = JSONDecoder()
```

2 สร้าง object JSONDecoder

```
let decoder = JSONDecoder()  
  
if let user = try? decoder.decode(User.self, from: jsonData) {  
    print(user.name) // จะแสดงผลเป็น "John Doe"  
}
```

2 เรียก function decode และใส่ข้อมูลที่เราต้องการจะแปลง


```
let decoder = JSONDecoder()  
  
if let user = try? decoder.decode(User.self, from: jsonData) {  
    print(user.name) // จะแสดงผลเป็น "John Doe"  
}
```

2 เรียก function decode และใส่ข้อมูลที่เราต้องการจะแปลง

```
let decoder = JSONDecoder()  
  
if let user = try? decoder.decode(User.self, from: jsonData) {  
    print(user.name) // จะแสดงผลเป็น "John Doe"  
}
```

2 เรียก function decode และใส่ข้อมูลที่เราต้องการจะแปลง

การ Map ชื่อจาก Key ของ JSON

```
{  
  "id": 1,  
  "name": "John Doe",  
  "email": "john.doe@example.com"  
}
```

```
struct User: Codable {  
  let id: Int  
  let name: String  
  let email: String  
}
```


การ Map ชื่อจาก Key ของ JSON

```
{  
  "id": 1,  
  "name": "John Doe",  
  "email": "john.doe@example.com"  
}
```

```
struct User: Codable {  
    let id: Int  
    let name: String  
    let email: String  
}
```

การ Map ชื่อจาก Key ของ JSON

```
{  
    "id": 1,  
    "name": "John Doe",  
    "email": "john.doe@example.com"  
}
```

```
struct User: Codable {  
    let id: Int  
    let name: String  
    let email: String  
}
```

การใช้ Map JSON ที่ชื่อไม่ตรง

```
{  
  "Id": 1,  
  "Name": "John Doe",  
  "Email": "john.doe@example.com"  
}
```

```
struct User: Codable {  
  let id: Int  
  let name: String  
  let email: String  
}
```


การใช้ Map JSON ที่ชื่อไม่ตรง

```
{  
  "Id": 1,  
  "Name": "John Doe",  
  "Email": "john.doe@example.com"  
}
```

```
struct User: Codable {  
    let id: Int  
    let name: String  
    let email: String
```

```
enum CodingKeys: String, CodingKey {  
    case id = "Id"  
    case name = "Name"  
    case email = "Email"  
}
```

```
}
```

การเรียก API ด้วย URLSession


```
func requestData() async {  
    guard let url = URL(string: "https://api.sample.com/") else { return }  
}
```

1 ใส่ URL ของ API ไปในตัวแปรประเภท URL

```
func requestData() async {  
    guard let url = URL(string: "https://api.sample.com") else { return }  
  
    do {  
  
    } catch {  
  
    }  
}
```

2 เพิ่มโค้ด do-catch เพื่อรองรับข้อผิดพลาดจากการเรียก API

```
func requestData() async {  
    guard let url = URL(string: "https://api.sample.com") else { return }  
  
    do {  
        let (data, _) = try await URLSession.shared.data(from: url)  
    } catch {  
  
    }  
}
```

3 เรียกใช้งาน URLSession และใส่ URL ใน parameter


```

func requestData() async {
    guard let url = URL(string: "https://api.sample.com") else { return }

    do {
        let (data, _) = try await URLSession.shared.data(from: url)

        let jsonDecoder = JSONDecoder()
        let apiData = try jsonDecoder.decode(Weather.self, from: data)
    } catch {
        print("Error: \(error)")
    }
}

```

3 แปลงข้อมูล JSON จาก API ด้วย JSONDecoder

```

func requestData() async {
    guard let url = URL(string: "https://api.sample.com") else { return }

    do {
        let (data, _) = try await URLSession.shared.data(from: url)

        let jsonDecoder = JSONDecoder()
        let apiData = try jsonDecoder.decode(Weather.self, from: data)
    } catch {
        print("Error: \(error)")
    }
}

```

3 แปลงข้อมูล JSON จาก API ด้วย JSONDecoder

```

func requestData() async {
    guard let url = URL(string: "https://api.sample.com") else { return }

    do {
        let (data, _) = try await URLSession.shared.data(from: url)

        let jsonDecoder = JSONDecoder()
        let apiData = try jsonDecoder.decode([Weather].self, from: data)
    } catch {
        print("Error: \(error)")
    }
}

```

3 แปลงข้อมูล JSON จาก API ด้วย JSONDecoder

เรียกข้อมูลจาก API ง่าย ๆ ใน 4 ขั้นตอน

- 1 ใส่ URL ของ API ไปในตัวแปรประเภท URL
- 2 เพิ่มโค้ด do-catch เพื่อรองรับข้อผิดพลาดจากการเรียก API
- 3 เรียกใช้งาน URLSession และใส่ URL ใน parameter
- 4 แปลงข้อมูล JSON จาก API ด้วย JSONDecoder

EXERCISE

Today Weather

ผ่าน Swift Playground บน iPad

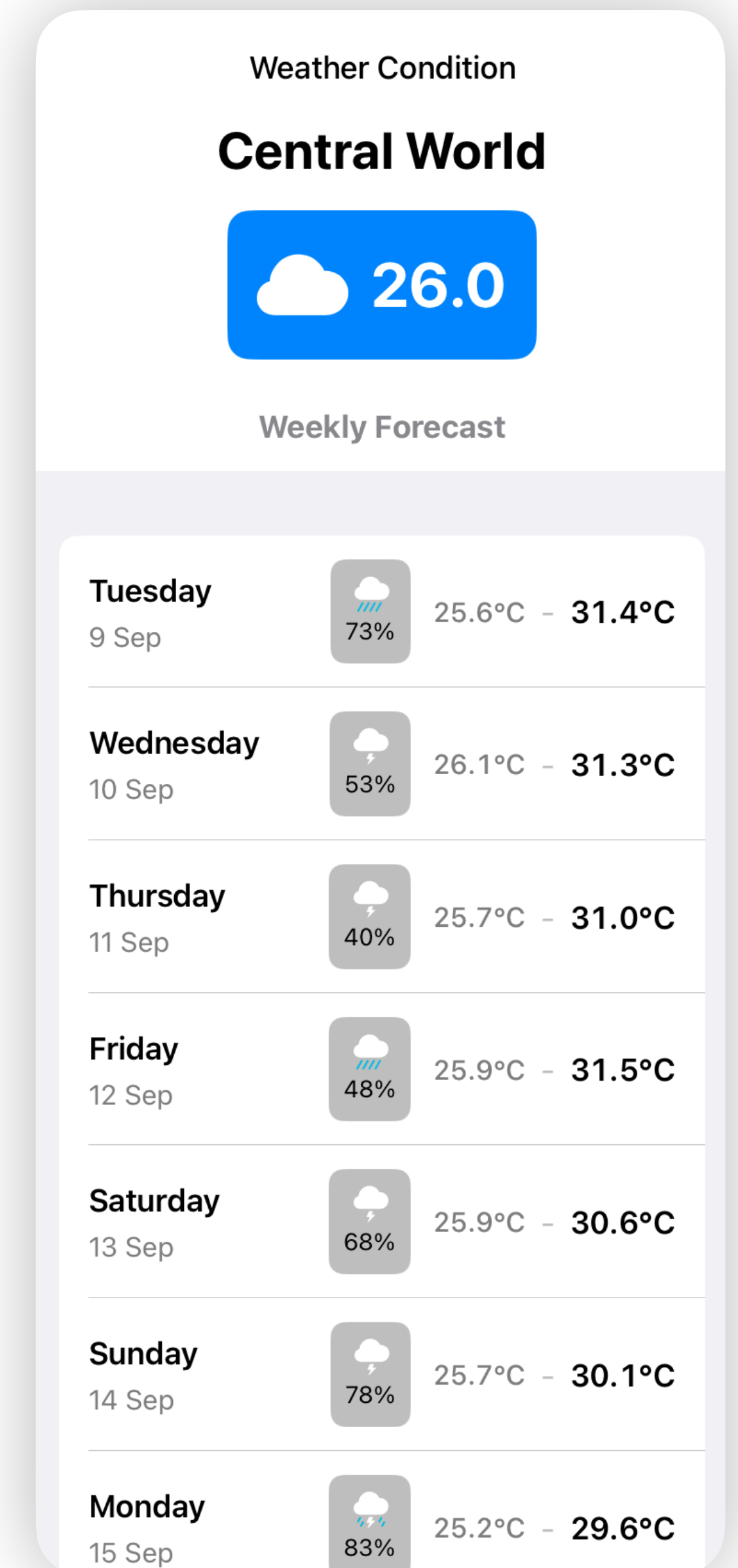


แบบฝึกหัด 4-3

Today Weather

สร้างแอป “Today Weather” สำหรับดึงและแสดงข้อมูลอากาศจาก **Open-Meteo API**

- มีการแบ่ง UI เป็น 2 ส่วน:
 - Current Condition:** แสดงชื่อสถานที่ (ตามที่กำหนดหรือระบุเอง) อุณหภูมิปัจจุบัน รูปสภาพอากาศ
 - Weekly Forecast:** แสดงรายการ 7 วัน โดยแสดง ชื่อวัน วันที่ รูปสภาพอากาศ โอกาสฝนตก อุณหภูมิต่ำสุดและสูงสุด (ชื่อวัน และวันที่สามารถใช้เป็น Text ที่เป็นค่าคงที่ได้)
- ใช้ **URLSession Codable** และ **CodingKeys** เพื่อแปลง JSON เป็น struct ที่จำเป็นต่อการแสดงผล
- เรียก API ด้วย URL https://api.open-meteo.com/v1/forecast?latitude=13.7467&longitude=100.5392&timezone=Asia/Bangkok¤t=temperature_2m,weather_code&daily=temperature_2m_max,temperature_2m_min,precipitation_probability_max,weather_code&forecast_days=7&timeformat=unixtime
- ★ **Tip:** ให้ลองเรียก URL นี้ใน Safari ก่อน เพื่อดูโครงสร้างของ JSON



แบบฝึกหัด 4-3

Today Weather

ข้อมูลอากาศปัจจุบัน

ข้อมูลของ 7 วันรวมวันนี้

```
{
  "latitude": 13.75,
  "longitude": 100.5,
  "generationtime_ms": 0.0746250152587891,
  "utc_offset_seconds": 25200,
  "timezone": "Asia/Bangkok",
  "timezone_abbreviation": "GMT+7",
  "elevation": 4,
  "current_units": {
    "time": "unixtime",
    "interval": "seconds",
    "temperature_2m": "°C",
    "precipitation": "mm",
    "weather_code": "wmo code"
  },
  "current": {
    "time": 1757358000,
    "interval": 900,
    "temperature_2m": 26,
    "precipitation": 0,
    "weather_code": 3
  },
  "daily_units": {
    "time": "unixtime",
    "weather_code": "wmo code",
    "temperature_2m_max": "°C",
    "temperature_2m_min": "°C",
    "precipitation_probability_max": "%"
  },
  "daily": {
    "time": [1757350800, 1757437200, 1757523600, 1757610000, 1757696400, 1757782800, 1757869200],
    "weather_code": [80, 95, 95, 80, 95, 95, 96],
    "temperature_2m_max": [31.4, 31.3, 31, 31.5, 30.6, 30.1, 29.6],
    "temperature_2m_min": [25.6, 26.1, 25.7, 25.9, 25.9, 25.7, 25.2],
    "precipitation_probability_max": [73, 53, 40, 48, 68, 78, 83]
  }
}
```

แบบฝึกหัด 4-3

Today Weather

Map Weather Code
เป็น SF Symbols

```
func mapWeatherCode(code: Int) -> String {  
    switch code {  
    case 0:  
        return "sun.max.fill" // Clear sky  
    case 1, 2, 3:  
        return "cloud.fill" // Clear/partly/overcast  
    case 45, 48:  
        return "cloud.fog.fill" // Fog  
    case 51, 53, 55:  
        return "cloud.drizzle.fill" // Drizzle  
    case 56, 57:  
        return "cloud.sleet.fill" // Freezing drizzle  
    case 61, 63, 65:  
        return "cloud.rain.fill" // Rain  
    case 66, 67:  
        return "cloud.hail.fill" // Freezing rain  
    case 71, 73, 75:  
        return "cloud.snow.fill" // Snow  
    case 77:  
        return "snow" // Snow grains  
    case 80, 81, 82:  
        return "cloud.heavyrain.fill" // Showers  
    case 85, 86:  
        return "cloud.snow.fill" // Snow showers  
    case 95:  
        return "cloud.bolt.fill" // Thunderstorm  
    case 96, 99:  
        return "cloud.bolt.rain.fill" // Thunderstorm with hail  
    default:  
        return "questionmark.circle" // Fallback  
    }  
}
```


แบบฝึกหัด 4-3

Today Weather

เพิ่มตัวแปรประเภท Date

```
struct Weather: Codable {  
    var temperature: Double  
    var date: Date  
}
```

ปรับค่าใน JSONDecoder ให้รองรับ
การแปลงวันที่แบบ Unix

```
let jsonDecoder = JSONDecoder()  
jsonDecoder.dateDecodingStrategy = .secondsSince1970
```